Stanislav Torchinskiy

Astana, Kazakhstan | +7 (705) 409-85-56 | torch0x.dev@gmail.com | github.com/torch0x

Education

Nazarbayev University — Astana, Kazakhstan

Bachelor of Science in Computer Science (Second-Year Undergraduate) — *Aug. 2025 – Expected Jun. 2029*

- **Relevant Coursework:** Performance & Data Structures, Computer Systems and Organization, Discrete Mathematics, Object-Oriented Programming, Programming Languages

Technical Skills

Smart Contracts & Protocols: Solidity, Foundry (Forge, Cast, Anvil), Hardhat, EVM Storage Layouts & Slot Packing, Gas Golfing / Opcode Optimization, EIP/ERC Standards (ERC-20, ERC-721, ERC-1155)

Programming Languages: Solidity, C/C++, Python, TypeScript, JavaScript, SQL, Bash

Web3 Libraries & Frontend: Ethers.js, Viem, React, Next.js, Node.js

Tools & Environments: Git, Linux, Docker

Projects

Aging Art (Dynamic On-Chain NFT Protocol) | *Solidity, ERC-721, On-Chain SVG, Base64, Foundry*

- Architected a fully on-chain, time-evolving ERC-721 collection with no IPFS or off-chain storage dependency by generating Base64-encoded SVG metadata directly in `tokenURI()` from `block.timestamp` and a per-token seed.
- Implemented a four-stage aging lifecycle (Blooming → Verdant → Mossy → Ancient) with an owner-triggered `care()` mechanic, 12-hour cooldown, and monotonic age-anchor invariant by tracking per-token action timestamps.
- Generated unique artwork per token (seed-derived tree geometry and six color palettes) by hashing token ID, minter, `blockhash`, and chain ID, and by splitting the SVG renderer into modular helpers with memory structs to avoid stack-too-deep errors.
- Enforced a hard supply cap and per-wallet mint limit that resist reentrancy bypass by ordering all checks and state updates before `_safeMint`; verified deterministic state transitions across multi-year time jumps with Forge integration and fuzz tests.

CPMM (Constant Product Automated Market Maker) | *Solidity, Foundry, OpenZeppelin, ERC-20*

- Built an on-chain AMM enforcing the $x \cdot y = k$ invariant, with a 0.3% swap fee accruing pro rata to liquidity providers and user-defined slippage bounds (`minAmountOut`, `minLiquidity`), by implementing Uniswap V2-style swap, mint, and burn math with ERC-20 LP shares in Solidity.
- Protected pool funds against reentrancy and first-depositor share manipulation by locking 1,000 minimum LP shares at a dead address, applying checks-effects-interactions ordering, and using `nonReentrant` and `SafeERC20`.
- Verified zero reserve drain under extreme edge cases across 10,000+ simulated transactions by building a Foundry test suite with stateful invariant tests and fuzz testing.
- Reduced gas overhead on swap and liquidity paths by caching reserves in memory to minimize SLOAD/SSTORE calls, and by using `immutable` token references and custom errors.

Sepolia Wallet Inspector (CLI) | *TypeScript, Node.js, Ethers.js, Alchemy API*

- Built an interactive command-line wallet inspector reporting network, chain ID, and ETH balance for any Ethereum or Sepolia address by querying an Alchemy JSON-RPC endpoint through Ethers.js v6.
- Retrieved wallet transfer history across external, internal, ERC-20, ERC-721, and ERC-1155 categories from genesis by calling Alchemy's `alchemy_getAssetTransfers` method.
- Prevented invalid input and misconfiguration by validating addresses with `ethers.isAddress`, loading the RPC URL from `.env`, and failing fast on missing configuration with typed error handling.

Certifications, Bootcamps & Hackathons

- **Cyfrin Updraft:** Certified in *Smart Contract Development* and *Foundry Fundamentals*, focused on fuzzing, invariant testing, and security auditing patterns.
- **Alchemy University:** Completed *Ethereum Developer Bootcamp* and *Learn JavaScript*, covering EVM core mechanics, Merkle proofs, and cryptographic signatures.
- **Decentrathon 4.0 Hackathon:** Competed in the inDrive AI track (team of 4) and developed an automated vehicle condition damage analysis computer vision pipeline under a strict 48-hour deadline.